

ROBOTIC ARM KINEMATICS & PATH PLANNING

Submitted By
~ Prakriti Das

Internship Program

DecodeLabs Robotics & Automation
Internship

Project Number

Week 1 – Robotic Arm Kinematics &
Path Planning

ABSTRACT

This project focuses on understanding robotic arm movement using inverse kinematics and path planning concepts. The objective is to calculate the required joint angles for a robotic arm to reach a target position accurately. The project demonstrates the basic principles of robotic motion control and positioning using Python.

OBJECTIVE

The objectives of this project are:

1. To understand robotic arm kinematics.
2. To calculate joint angles using inverse kinematics.
3. To determine the position of a robotic arm end effector.
4. To demonstrate basic robotic path planning.
5. To simulate robotic movement using Python.

TOOLS USED

- . Python
- . Google Colab
- . Math Library

INTRODUCTION

Robotic arms are widely used in manufacturing, assembly lines, and industrial automation systems. To perform tasks accurately, the robot must calculate the correct joint angles required to reach a target position. This process is known as inverse kinematics. Path planning is used to determine how the robotic arm moves from one position to another in a smooth and efficient manner.

METHODOLOGY

Step 1: Define Arm Parameters

The robotic arm is represented using two links with predefined lengths.

Step 2: Define Target Position

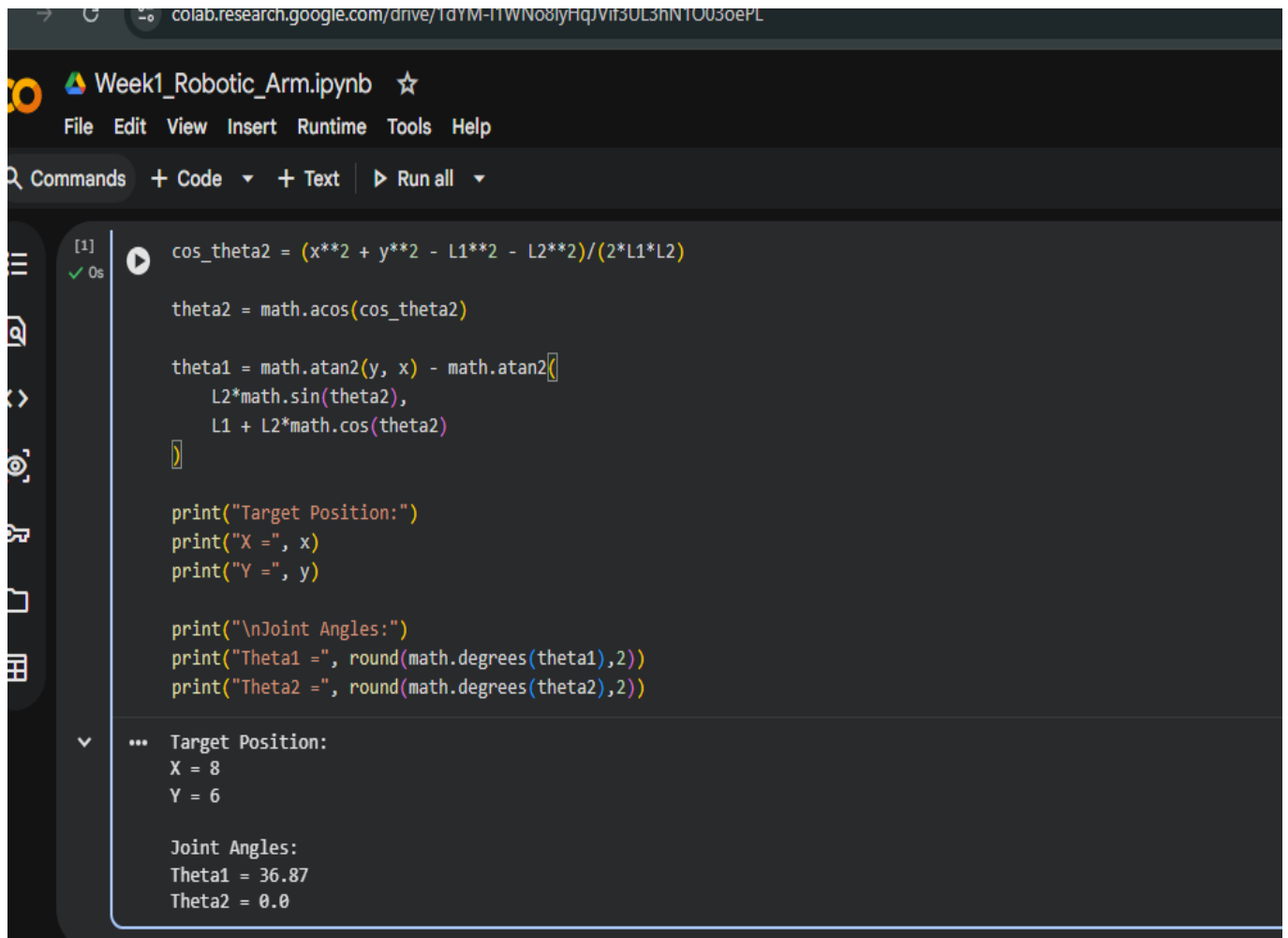
A target coordinate (X,Y) is selected for the robotic arm.

Step 3: Apply Inverse Kinematics

Mathematical equations are used to calculate the required joint angles for reaching the target position.

Step 4: Display Results

The calculated joint angles are displayed as the final output.



The screenshot shows a Google Colab notebook titled "Week1_Robotic_Arm.ipynb". The code in the first cell calculates the joint angles for a two-link robotic arm given target coordinates (x=8, y=6). The code uses the law of cosines to find the second joint angle (theta2) and then the law of sines to find the first joint angle (theta1). The output displays the target position and the calculated joint angles.

```
[1] ✓ 0s cos_theta2 = (x**2 + y**2 - L1**2 - L2**2)/(2*L1*L2)

theta2 = math.acos(cos_theta2)

theta1 = math.atan2(y, x) - math.atan2(
    L2*math.sin(theta2),
    L1 + L2*math.cos(theta2)
)

print("Target Position:")
print("X =", x)
print("Y =", y)

print("\nJoint Angles:")
print("Theta1 =", round(math.degrees(theta1),2))
print("Theta2 =", round(math.degrees(theta2),2))
```

... Target Position:
X = 8
Y = 6

Joint Angles:
Theta1 = 36.87
Theta2 = 0.0

RESULTS

The robotic arm successfully calculated:

- Target position coordinates
- Joint Angle 1 (Theta1)
- Joint Angle 2 (Theta2)

The calculated values demonstrate how inverse kinematics can be used to control robotic arm movement.

APPLICATIONS

- Industrial Robotics
- Pick and Place Systems
- Manufacturing Automation
- CNC Machines
- Warehouse Automation

ADVANTAGES

1. Accurate positioning of the robotic arm.
2. Efficient calculation of joint angles using inverse kinematics.
3. Improved movement precision and control.
4. Reduced human effort in industrial operations.
5. Supports automation in manufacturing and assembly processes.
6. Enables smooth and optimized path planning.
7. Can be extended for advanced robotic applications.

CONCLUSION

The project successfully demonstrated the basic concepts of robotic arm kinematics and inverse kinematics. By calculating the required joint angles for a target coordinate, the robotic arm can accurately position its end effector. This project provides a foundation for advanced robotic manipulation and path planning systems.

REFERENCES

1. Python Documentation
2. Robotics Fundamentals
3. Inverse Kinematics Theory
4. DecodeLabs Robotics & Automation Training Material